
🚦 What Is Node Affinity?

📘 Definition:

Node Affinity allows you to **control which nodes your pods are allowed (or preferred) to run on**, based on **node labels**.

It's like saying:

"Hey Kubernetes, I want my app to run only on certain types of nodes — not just anywhere!"

✅ Types of Node Affinity:

1. `requiredDuringSchedulingIgnoredDuringExecution` (🔒 **Hard rule**)
 - Pod **will NOT be scheduled** unless this is met.
2. `preferredDuringSchedulingIgnoredDuringExecution` (🎯 **Soft rule**)
 - Pod **prefers** the condition but will still run elsewhere if needed.

👤 Our Example — Node Affinity (inverted):

In this part of the YAML:

```
affinity:
```

```
  nodeAffinity:
```

```
    requiredDuringSchedulingIgnoredDuringExecution:
```

```
      nodeSelectorTerms:
```

```
        - matchExpressions:
```

- key: nodeName
- operator: NotIn
- values:
 - node1

What's Happening?

- This says: “Do not schedule pods on nodes labeled **nodeName=node1.**”
 - So if you labeled:
 - **i-00ea164f7b5a58e08** as **nodeName=node1**
 - Kubernetes will **not schedule any pod here.**
 - It will only look at other nodes (like **node2** or unlabeled ones).
-

What Is Pod Anti-Affinity?

Definition:

Pod Anti-Affinity restricts **where your pods can run relative to other pods.** It says:

"Do not schedule this pod **on the same node as another pod** with certain labels."

Our Example — Pod Anti-Affinity:

podAntiAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

matchExpressions:

- key: app
operator: In
values:
- my-app
topologyKey: kubernetes.io/hostname

What's Going On?

- This says: “**Do not run this pod on the same node as any other pod with app: my-app.**”
 - So if 1 pod is already on **node2**, the next pod will try to go to **another node** (like **node3**).
 - **topologyKey: kubernetes.io/hostname** tells K8s to enforce this **at node level** (one hostname = one node).
-

Full Real-Life Behavior

Let's say you deploy this:

replicas: 3

You have 3 nodes:

- **i-00ea164f7b5a58e08** labeled **nodeName=node1**
- **i-060932a734e69d5e7** labeled **nodeName=node2**
- Another unlabeled node

Here's What Happens:

1. **Kubernetes scheduler wakes up:** "I need to place 3 pods."
2. It sees `node1` → ❌ **excluded** (due to nodeAffinity NotIn)
3. It sees `node2` → ✅ eligible!
4. First pod: 🎉 goes to `node2`
5. Second pod: ❌ can't go to `node2` (same `app=my-app` already running) → tries another node
6. Third pod: ❌ avoids both `node2` and `node1` → lands on unlabeled node (if available)

If you have only one eligible node, the rest of the pods will remain in **Pending state**.

Summary Table

Feature	What It Controls	Based On	Example Used
Node Affinity	Which nodes a pod can run on	Node labels	Exclude <code>nodeName=node1</code>
Pod Anti-Affinity	Where pods run relative to others	Pod labels	Avoid scheduling <code>my-app</code> on same node

Pro Tip for Real Scenarios:

- Use **node affinity** to target hardware or zones: `gpu=true`, `zone=us-east-1b`, etc.

- Use **anti-affinity** to **spread replicas** across nodes for **HA (High Availability)**.
 - Combine both to make intelligent placement decisions.
-